编译技术实验课

主讲教师：陈安龙

课程实验内容简介

实验一：词法分析 （4学时）

用C语言编写程序实现：1）识别给定程序中所有的整数、浮点数及字符串常量；2) 识别给定程序中所有的符号、标识符及关键字；用 flex 完成上述任务；

实验上传内容：

① 请在Icoding作业页面上传词法分析实验所有源代码及实验报告；

② 源代码至少包括一个flex源文件，即 .l 文件，如有其它 .c 或 .h 文件，也请一并上传；

③ 注：所有源代码及实验报告需同时上传，需同时上传，需同时上传。

④ 所有文件都以 utf-8 进行统一编码保存！！！

⑤ 检查环境为Linux，如出现乱码等错误，将扣分！

⑥ 实验报告：点名册序号-词法分析_姓名.docx

第一次实验内容：

请根据给定的文法，利用flex设计一词法分析器，该分析器从标准输入读入源代码后，输出单词的类别编号及附加信息。 附加信息规定如下：

① 当类别为 Y_ID、num_INT 或 num_FLOAT 时，附加信息为该类别对应的属性，如 main, 100, 29.3等；

② 当类别为 关键字 时，附件信息为 KEYWORD；

③ 当类别为 运算符 时，附件信息为 OPERATOR；

④ 当类别为 其它符号时，附件信息为 SYMBOL；

例如对于源代码：

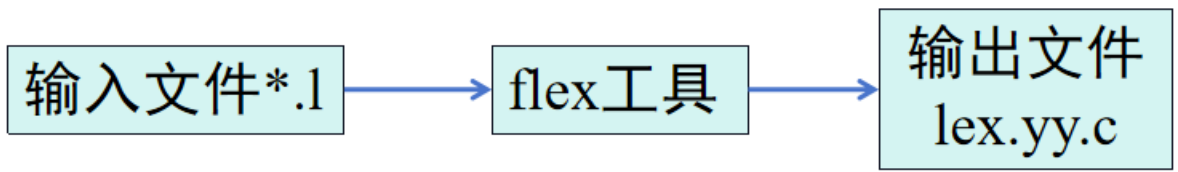
```
int main(){
    return 3;
}
```

词法分析器的输出为：

```
<261, KEYWORD>
<260, main>
<285, SYMBOL>
<286, SYMBOL>
<287, SYMBOL>
<269, KEYWORD>
<258, 3>
<292, SYMBOL>
<288, SYMBOL>
```

1. FLEX简介

单词的描述称为模式(Lexical Pattern)，模式一般用正规表达式进行精确描述。**FLEX通过读取一个有规定格式的文本文件，输出一个如下所示的C语言源程序。**



FLEX的输入文件称为LEX源文件，**它内含正规表达式和对相应模式处理的C语言代码**。LEX源文件的扩展名习惯上用.l表示。FLEX通过对源文件的**扫描自动生成相应的词法分析函数 int yylex()**，并将之输出到名规定为lex.yy.c的文件中。实用时，可将其改名为lexyy.c。**该文件即为LEX的输出文件或输出的词法分析器。**也可将 int yylex()加入自己的工程文件中使用。

本实验中的单词类别定义如下，该定义包含在 token.h 中，此头文件可直接使用，无需上传

```
enum yytokentype {
        num_INT = 258,
        num_FLOAT = 259,
        Y_ID = 260,
        Y_INT = 261,
        Y_VOID = 262,
        Y_CONST = 263,
        Y_IF = 264,
        Y_ELSE = 265,
        Y_WHILE = 266,
        Y_BREAK = 267,
        Y_CONTINUE = 268,
        Y_RETURN = 269,
        Y_ADD = 270,
        Y_SUB = 271,
        Y_MUL = 272,
        Y_DIV = 273,
        Y_MODULO = 274,
        Y_LESS = 275,
        Y_LESSEQ = 276,
        Y_GREAT = 277,
        Y_GREATEQ = 278,
        Y_NOTEQ = 279,
        Y_EQ = 280,
        Y_NOT = 281,
        Y_AND = 282,
        Y_OR = 283,
        Y_ASSIGN = 284,
        Y_LPAR = 285,
        Y_RPAR = 286,
        Y_LBRACKET = 287,
        Y_RBRACKET = 288,
        Y_LSQUARE = 289,
        Y_RSQUARE = 290,
        Y_COMMA = 291,
        Y_SEMICOLON = 292,
        Y_FLOAT = 293
};
```

实验用软件

https://sourceforge.net/projects/tdm-gcc/files/

https://sourceforge.net/projects/gnuwin32/files/flex/2.5.4a-1/

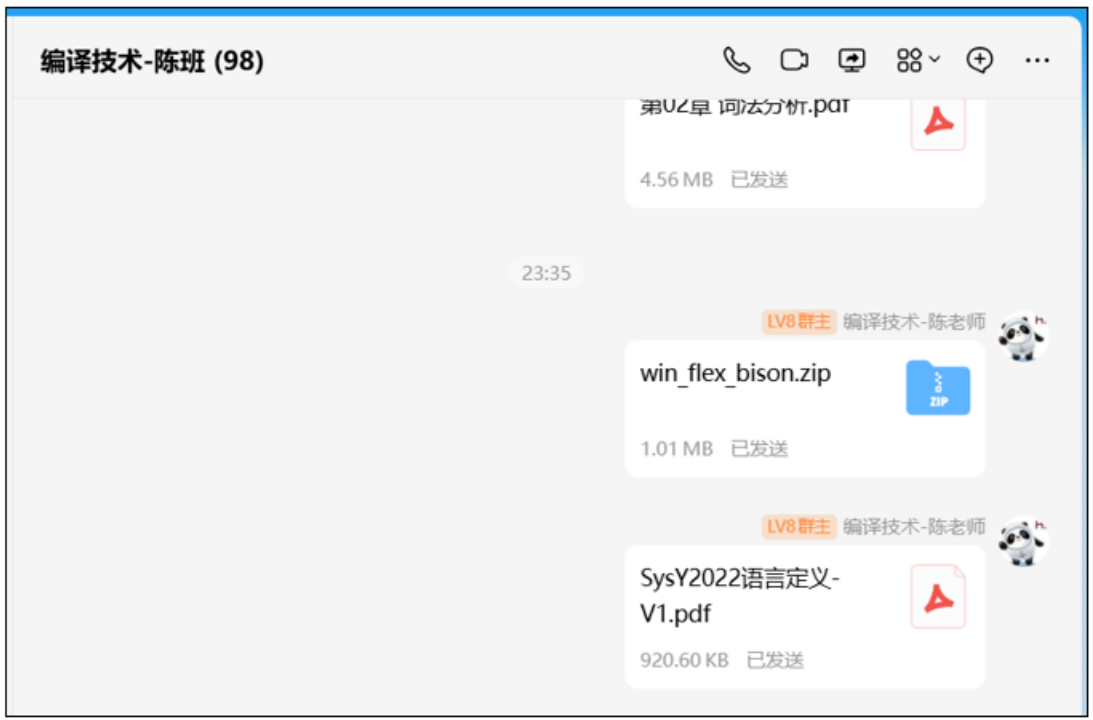
https://sourceforge.net/projects/gnuwin32/files/bison/2.4.1/

https://sourceforge.net/projects/winflexbison/

在QQ群里已经有flex2.6.4 和bison3.8.2，大家在群里自行下载

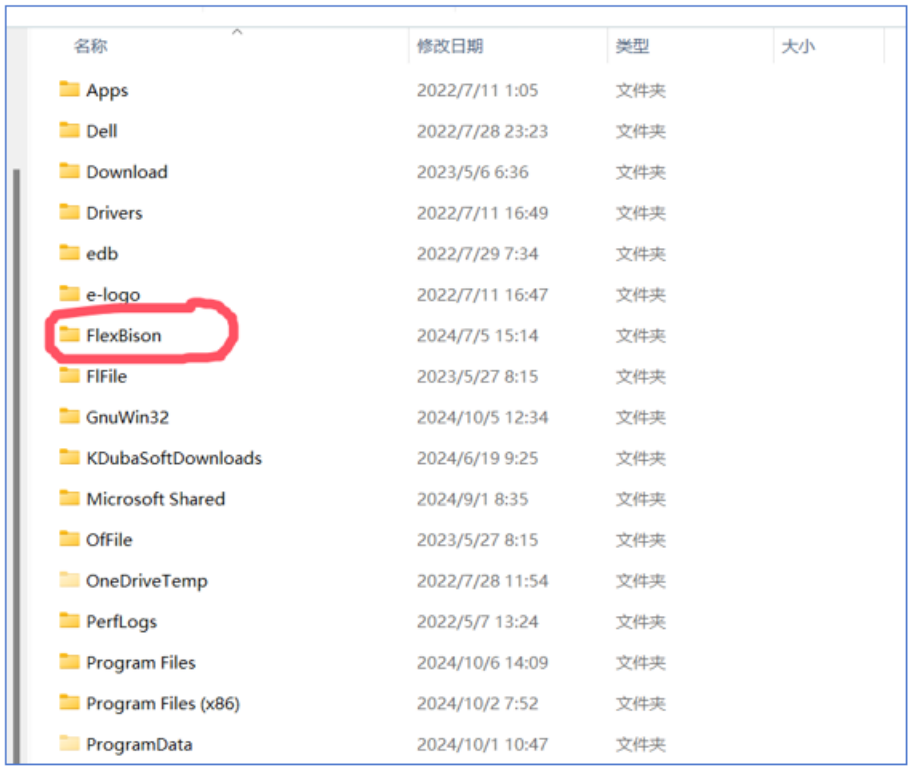
2. FLEX运行环境

在QQ群里下载FlexBison.zip文件，这里有flex2.6.4和bison3.8



解压FlexBison.zip文件

将FlexBison.zip文件解压到C:盘根目录，生成文件目录为FlexBison

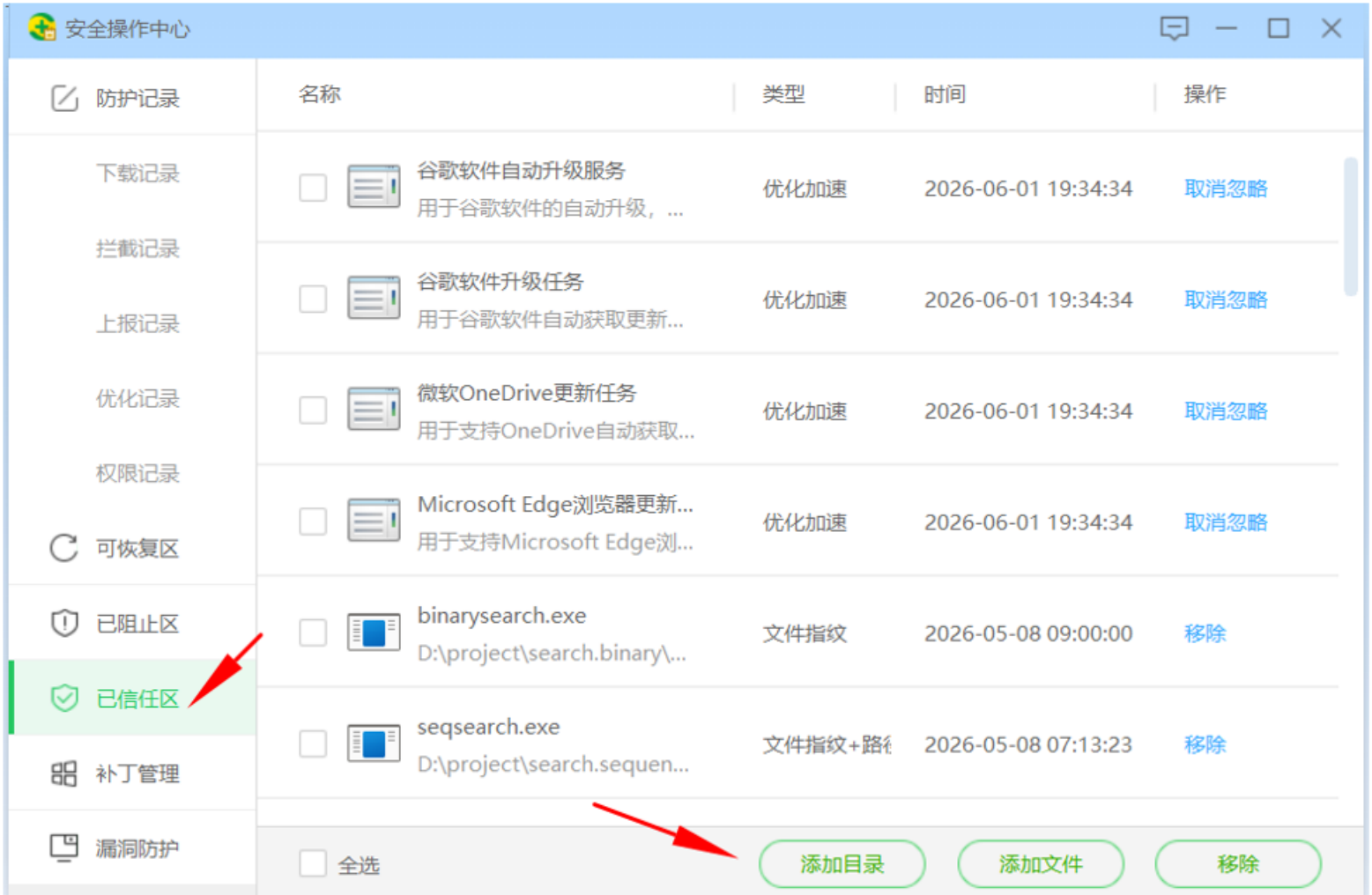




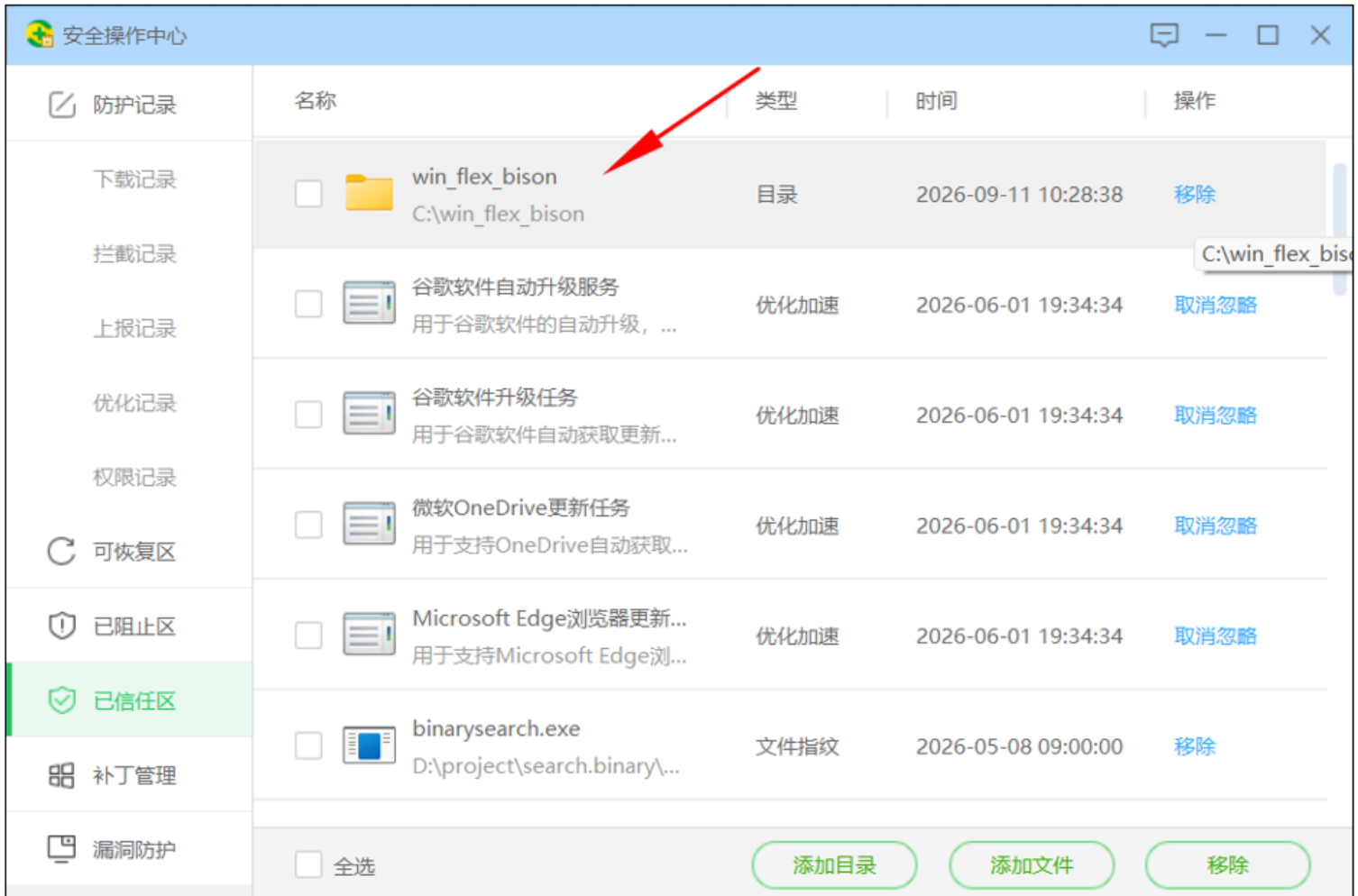
将目录C:\FlexBison\bin添加到360的白名单中，避免误认为病毒



将目录C:\FlexBison\bin添加到360的白名单中，避免误认为病毒

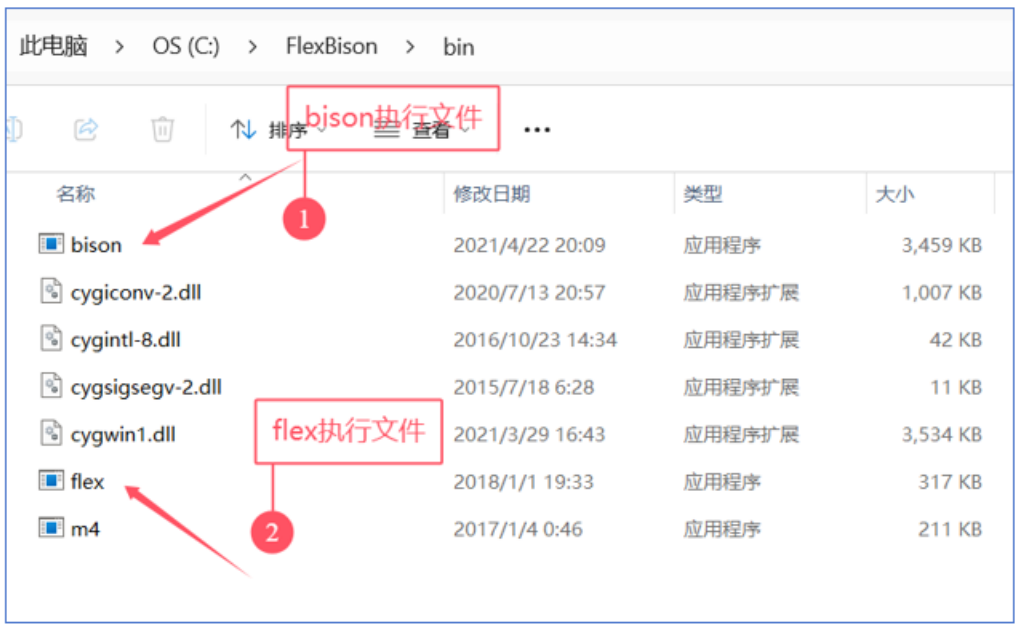


将目录C:\FlexBison\bin添加到360的白名单中，避免误认为病毒



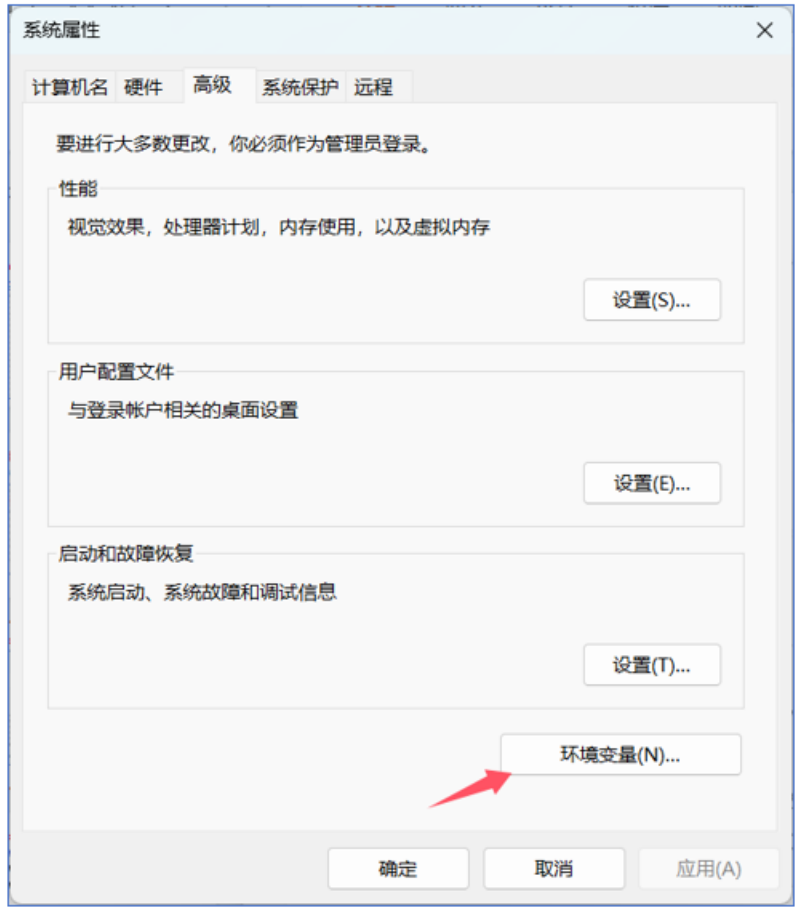
Flex命令文件目录

执行文件所在目录C:\FlexBison\bin



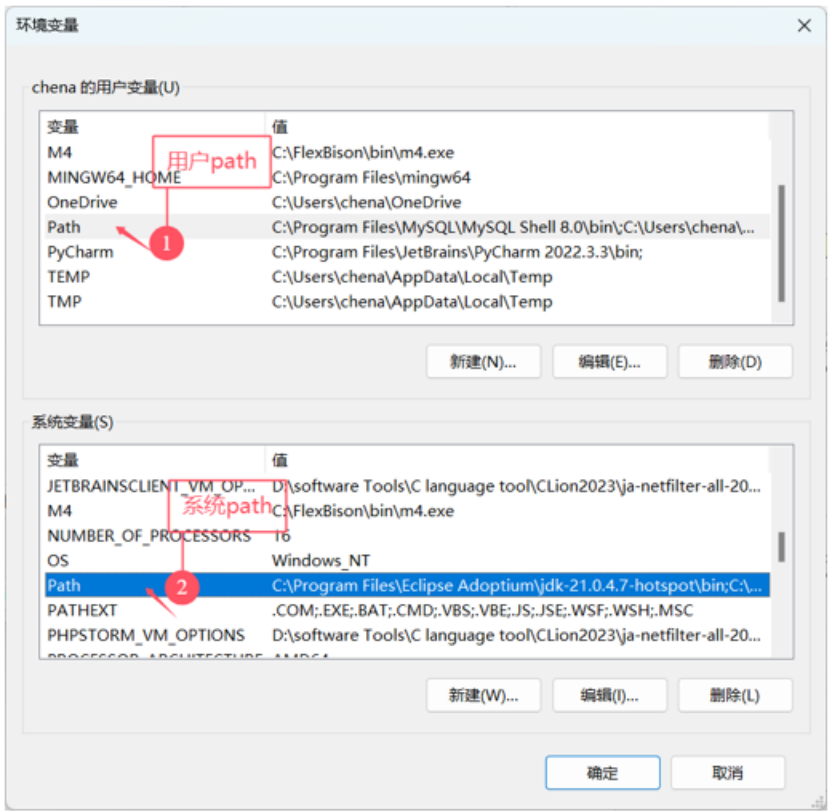
配置Flex&bison环境变量

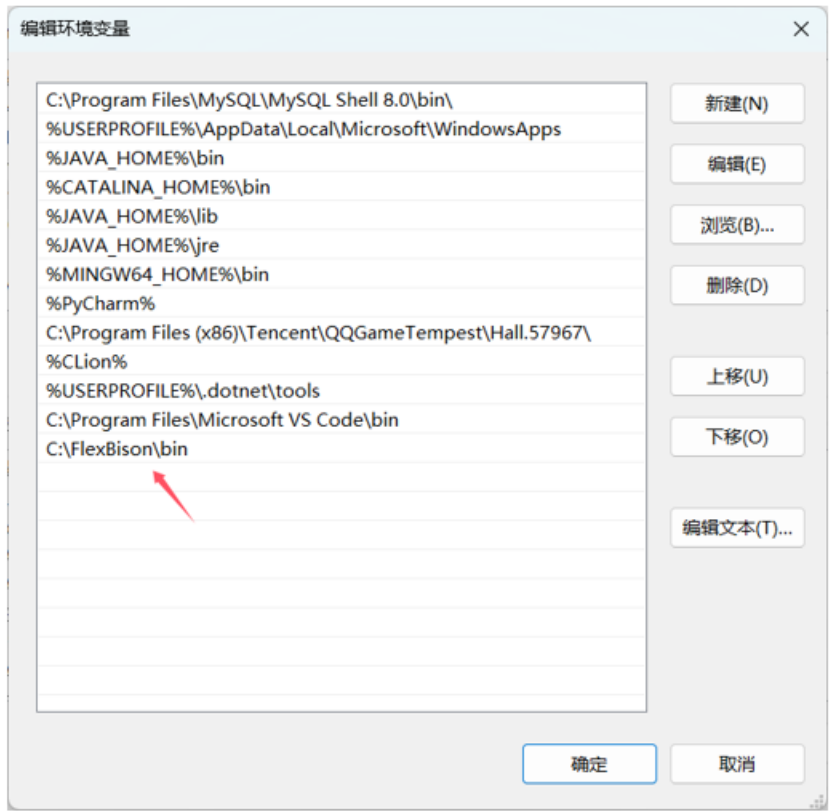




配置Flex&bison环境变量

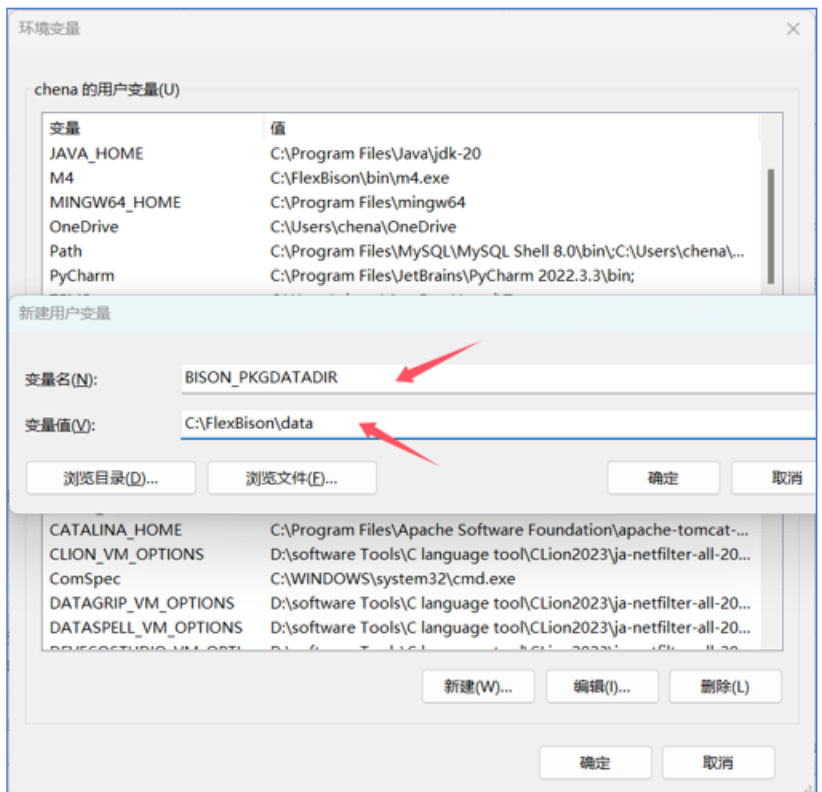
1.修改用户/系统的环境变量，将C:\FlexBison\bin加到PATH中

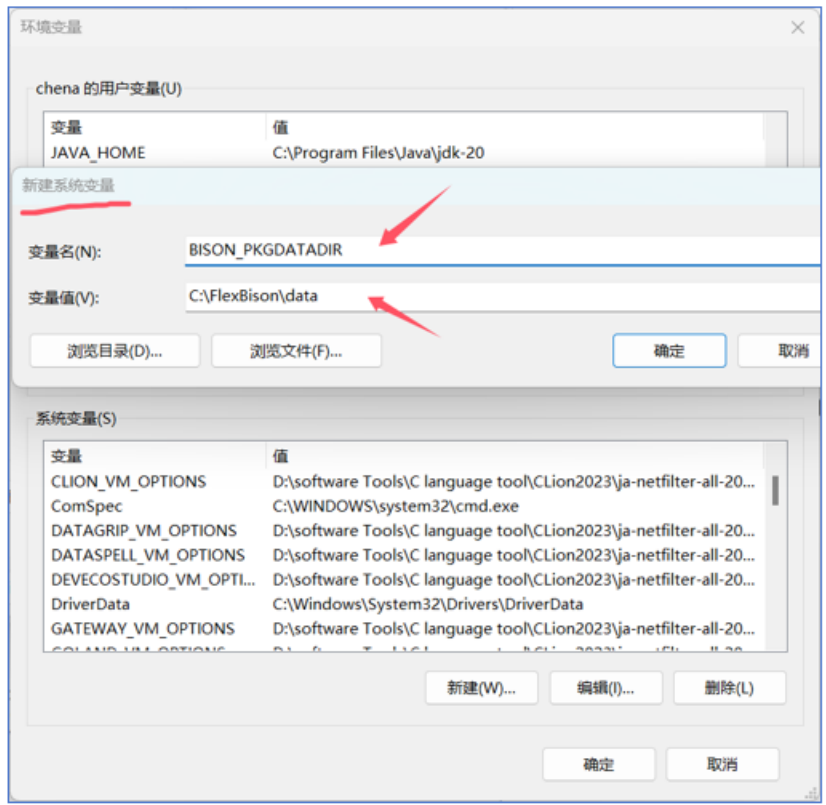




配置Flex&bison环境变量

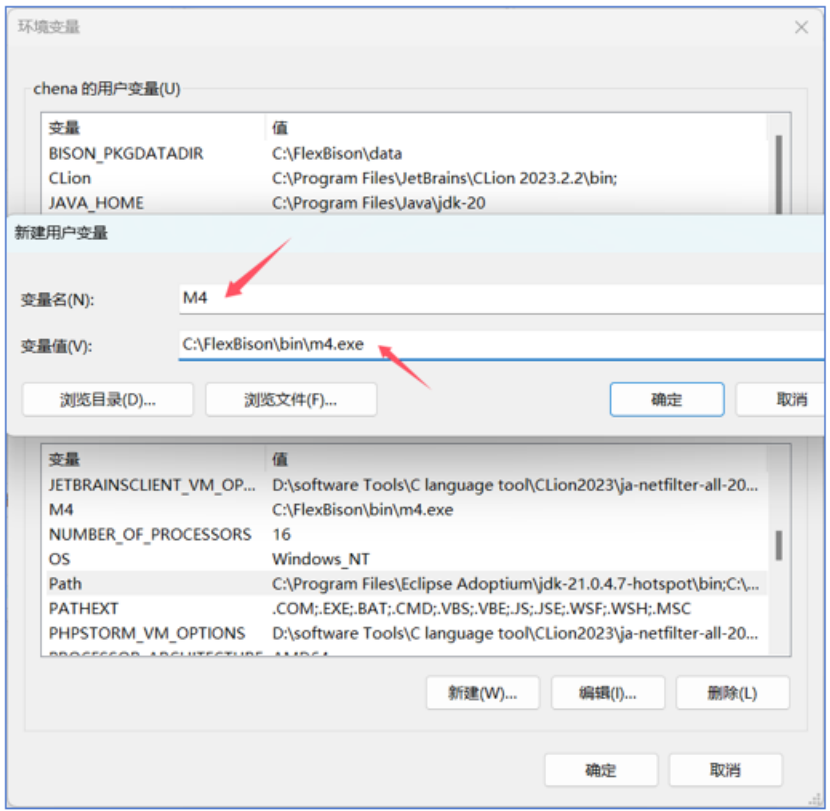
2.增加用户/系统的环境变量BISON_PKGDATADIR, 将值设置为C:\FlexBison\data

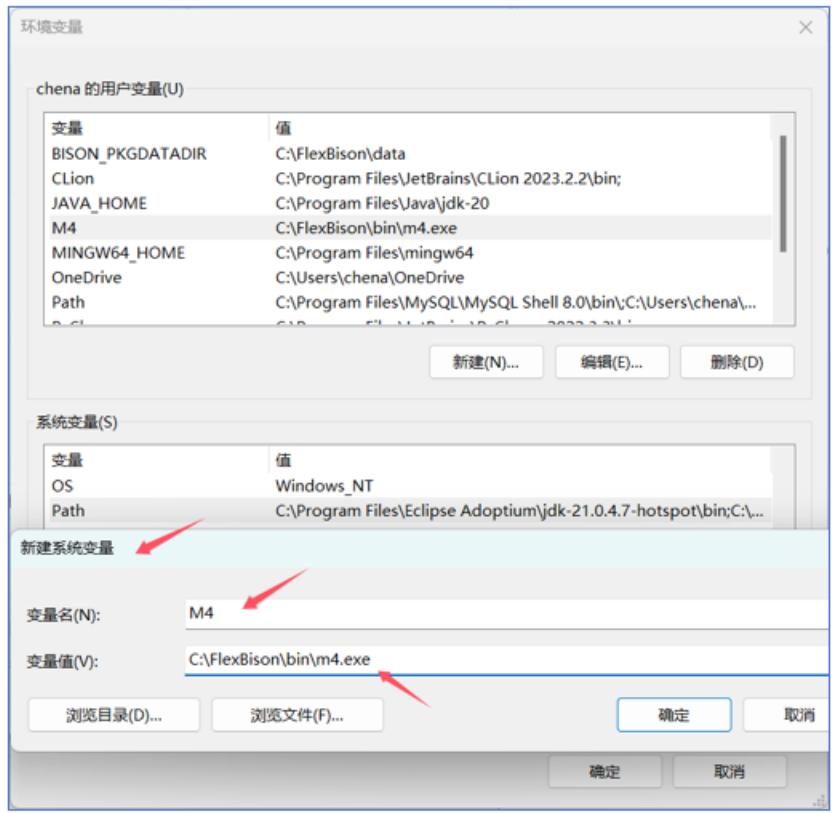




配置Flex&bison环境变量

3.增加用户/系统的环境变量M4，将值设置为C:\FlexBison\bin\m4.exe





定义部份

模式的宏定义部份如同C语言中的宏定义，通过宏名定义一个模式，这样，可以简化在源文件中多次出现的正规表达式的书写。格式为：

宏名1 宏定义1

宏名2 宏定义2

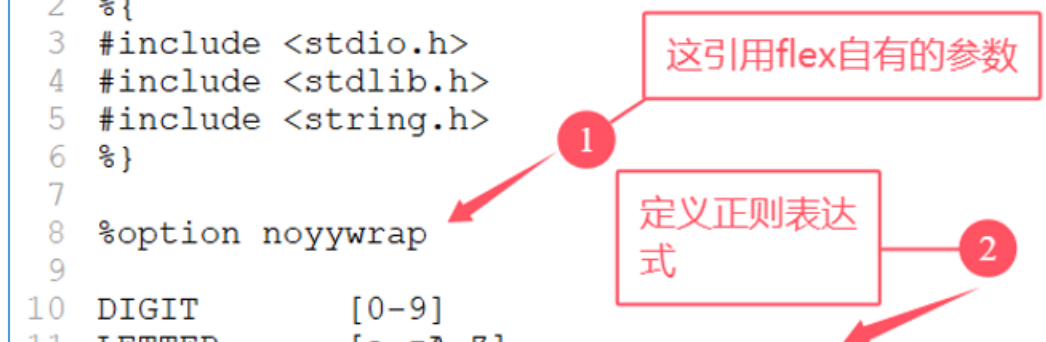
……

例如：

DIGIT [0-9]

ID [A-Za-z][A-Za-z0-9_]*

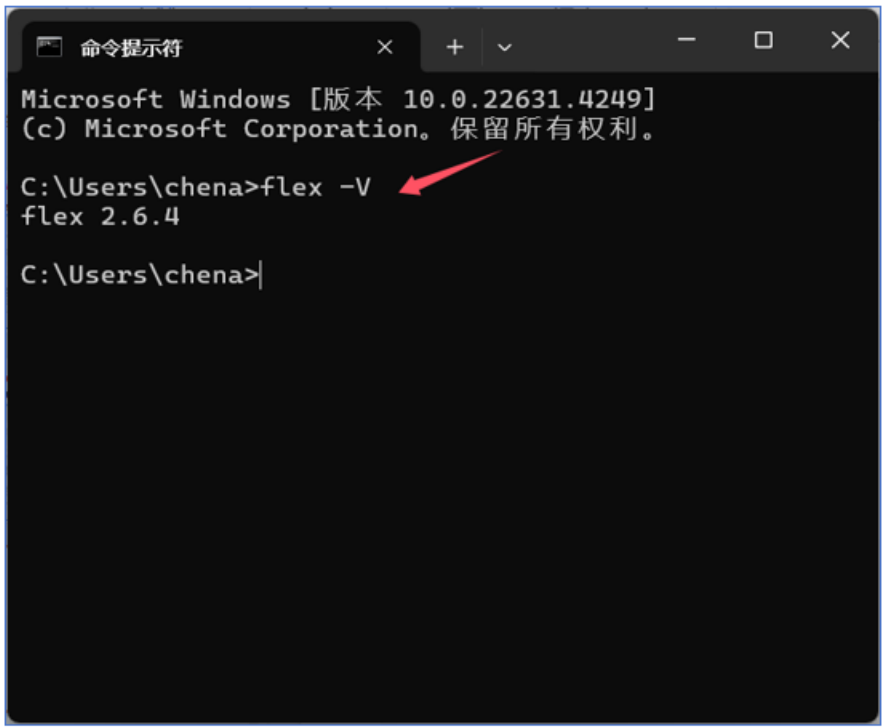
```
1
2  %{
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <string.h>
6  %}
7
8  %option noyywrap
9
10 DIGIT          [0-9]
11 LETTER         [a-zA-Z]
12 ID             {LETTER}({LETTER}|{DIGIT})*
13 NUMBER         {DIGIT}+(\.{DIGIT}+)?
14
```



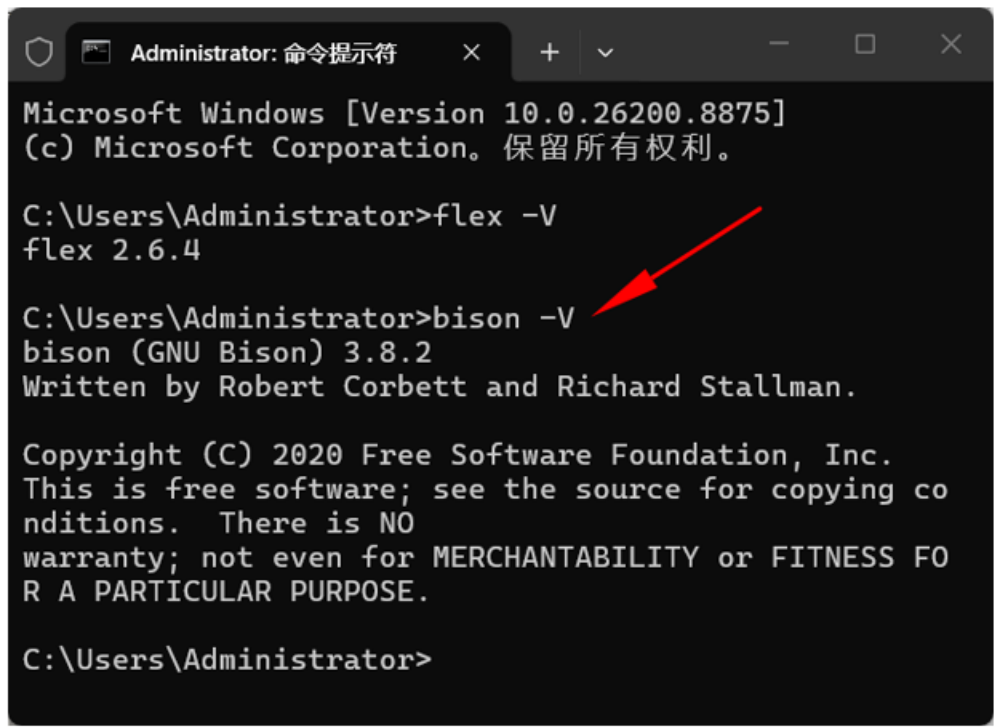
宏名是以字母和下划线”_”开始，以字母、数字和下划线组成的字符串，且大小写敏感。宏名必须顶行写，宏名和宏定义必须写在同一行上。宏名和宏定义之间以空格符或TAB符隔开。

测试环境

C:>flex -V



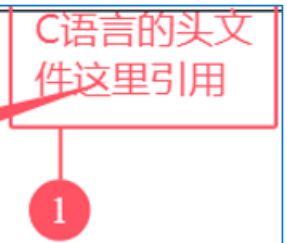
C:>bison -V



定义部份

定义部份由C语言代码、模式的宏定义、条件模式的开始条件说明三部份组成。其中，C代码部份由顶行的%{和}%引入，LEX扫描源文件时将%{和}%之间的部分原封不动的拷贝到输出文件*.c中.

```
1
2  %{
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <string.h>
6  %}
7
```



Flex定义正则表达式

1、简单字符类

① [abc] 匹配 'a', 'b' 或 'c'

② [^abc] 匹配任何不是 'a','b' 或 'c' 的字符

③ [a-z] 匹配任意小写字母

④ [A-Z] 匹配任意大写字母

⑤ [0-9] 匹配任意数字

⑥ [a-zA-Z] 匹配任意字母

Flex定义正则表达式（续）

2、预定义字符类

① \d 匹配数字，等同于 [0-9]

② \D 匹配非数字，等同于 [^0-9]

③ \w 匹配单词字符（字母、数字、下划线），等同于 [a-zA-Z0-9_]

④ \W 匹配非单词字符

⑤ \s 匹配空白字符（空格、制表符、换行等）

⑥ \S 匹配非空白字符

⑦ . 匹配除换行符外的任意字符

Flex定义正则表达式（续）

3、重复操作符

① * 零次或多次匹配

② + 一次或多次匹配

③ ? 零次或一次匹配

④ {n} 恰好 n 次匹配

⑤ {n,} 至少 n 次匹配

⑥ {n,m} n 到 m 次匹配

① a* 空串、a、aa、aaa...

② [0-9]+ 一个或多个数字

③ [+-]?[0-9]+ 可选符号后跟数字

④ [0-9]{4} 恰好四位数字

⑤ [0-9]{2,4} 二到四位数字

⑥ [0-9]{3,} 至少三位数字

⑦ ab* 等价于 a(b*)，不是 (ab)*

⑧ ab|cd* 等价于 ab | (cd*)

⑨ (ab)+ 匹配 ab、abab、ababab...

⑩ (ab|cd)e 匹配 abe 或 cde

⑪ a(bc)*d 匹配 ad、abcd、abcbcd...

⑫ ((a|b)c)+ 匹配 ac、bc、acbc、acac...

⑬ a(b|c)d 匹配 abd 或 acd

⑭ cat|dog 匹配 cat 或 dog

⑮ a|b|c 匹配 a、b 或 c

Flex定义正则表达式（续）

4、位置锚点

① ^ 匹配行首

② $ 匹配行尾

③ \b 匹配单词边界

④ \< 匹配单词开头

⑤ \> 匹配单词结尾

[^a-z]　匹配任意非小写字母的字符

[^\n]　匹配除换行外的任意字符

;$　{ /* 行尾的分号 */ }

例如：

int x;　← 匹配分号

int y;　← 匹配分号

; int z　← 不匹配（分号不在行尾）

[0-9]+$　{ /* 行尾的数字序列 */ }

abc123　← 匹配 123

abc123def　← 不匹配

Flex定义正则表达式（续）

5、特殊转义序列

① \n 换行符

② \t 制表符

③ \r 回车符

④ \f 换页符

⑤ \0 空字符

⑥ \xHH 十六进制字符

⑦ \ooo 八进制字符

例子：

① *　匹配 '*'

② \+　匹配 '+'

③ \?　匹配 '?'

④ \(\)　匹配括号

⑤ \[\]　匹配方括号

⑥ \{ \}　匹配花括号

⑦ \|　匹配 '|'

⑧ \^　匹配 '^'

⑨ \$　匹配 '$'

⑩ \.　匹配 '.'

⑪ \/　匹配 '/'

Flex定义正则表达式（续）

6、带引号

① "if"　　匹配字面量 if

② "a+b"　　匹配字面量 a+b，而不是 a 重复一次再跟 b

③ "*"　　匹配字面量星号

④ "{"　　匹配字面量左花括号

⑤ "if"|"else"|"while"　　匹配三个关键字之一

⑥ "0x"[0-9a-fA-F]+　　匹配十六进制数

定义程序的token标记符示例

```
%{
    enum yytokentype{
          INT=256,
          IF=257,
          ELSE=258,
          WHILE=259,
          RETURN=260,
          BREAK=261,
          CONTINUE=262,
          ASSIGN=263,
          EQ=264,
          NEQ=265,
          LT=266,
          LE=267,
          GT=268,
          GE=269,
          ADD=270,
          SUB=271,
          MUL=272,
          DIV=273,
          MOD=274,
          LPAREN=275,
          RPAREN=276,
          LBRACE=277,
          RBRACE=278,
          SEMICOLON=279,
          NUMBER=280,
          ID=281,
     };
    int yylval;
    FILE *yyin; // 文件输入流
%}
```





规则定义部份

规则部份是FLEX源文件的核心部份，它包括一组模式和在生成分析器识别相应模式后对相应模式进行处理的C语言动作(Action)。格式如下

C语言代码

模式1 动作1

模式2 |

模式3 动作3

……

同定义部分一样，C语言代码必须出现在第一个模式之前，包括在%{和}%之中，且%{必须顶行书写。%{和}%之间的代码部份可用来定义yylex()用到的局部变量。

模式必须顶行书写。模式可为正规式或用{}括起且在定义部份定义过的宏名。动作为用{}括起的C代码。且开始括号{与模式之间用白字符隔开，且须和模式在同一行上。注意，在模式后加一|表示模式2和3采用同一动作3.|和模式2以白字符隔开。

在%% ...%%之间定义标记符分析的子程序片段

```
47
48  %%
49  "int"         { return INT; }
50  "if"          { return IF; }
51  "else"        { return ELSE; }
52  "while"       { return WHILE; }
53  "return"      { return RETURN; }
54  "break"       { return BREAK; }
55  "continue"    { return CONTINUE; }
56  "="           { return ASSIGN; }
57  "=="          { return EQ; }
58  "!="          { return NEQ; }
59  "<"           { return LT; }
60  "<="          { return LE; }
61  ">"           { return GT; }
62  ">="          { return GE; }
63  "+"           { return ADD; }
64  "-"           { return SUB; }
65  "*"           { return MUL; }
66  "/"           { return DIV; }
67  "%"           { return MOD; }
68  "("           { return LPAREN; }
69  ")"           { return RPAREN; }
70  "{"           { return LBRACE; }
71  "}"           { return RBRACE; }
72  ";"           { return SEMICOLON; }
73  {ID}          { return ID; }
74  {NUMBER}      { yylval = atof(yytext); return NUMBER; }
75  [ \t\n]       ; // 忽略空格、制表符和换行符
76  .             { printf("ERROR: Invalid character %s\n", yytext); }
77  %%
78
```

定义词法分析分析标记符的动作子程序片段

1

在%% ...%%之间定义标记符分析的子程序片段

```
47
48  %%
49  "int"         { return INT; }
50  "if"          { return IF; }
51  "else"        { return ELSE; }
52  "while"       { return WHILE; }
53  "return"      { return RETURN; }
54  "break"       { return BREAK; }
55  "continue"    { return CONTINUE; }
56  "="           { return ASSIGN; }
57  "=="          { return EQ; }
58  "!="          { return NEQ; }
59  "<"           { return LT; }
60  "<="          { return LE; }
61  ">"           { return GT; }
62  ">="          { return GE; }
63  "+"           { return ADD; }
64  "-"           { return SUB; }
65  "*"           { return MUL; }
66  "/"           { return DIV; }
67  "%"           { return MOD; }
68  "("           { return LPAREN; }
69  ")"           { return RPAREN; }
70  "{"           { return LBRACE; }
71  "}"           { return RBRACE; }
72  ";"           { return SEMICOLON; }
73  {ID}          { return ID; }
74  {NUMBER}      { yylval = atof(yytext); return NUMBER; }
75  [ \t\n]       ; // 忽略空格、制表符和换行符
76  .             { printf("ERROR: Invalid character %s\n", yytext); }
77  %%
78
```

定义词法分析分析标记符的动作子程序片段

1

用户附加C语言部份

FLEX对此部份不作任何处理，仅仅将之直接拷贝到输出文件*.c的尾部。在些部份，可定义对模式进行处理的C语言函数、主函数和yylex要调用的函数yywrap()等。如果用户在其它C模块中提供这些函数，用户代码部份可以省略。

编写C语言的main程序

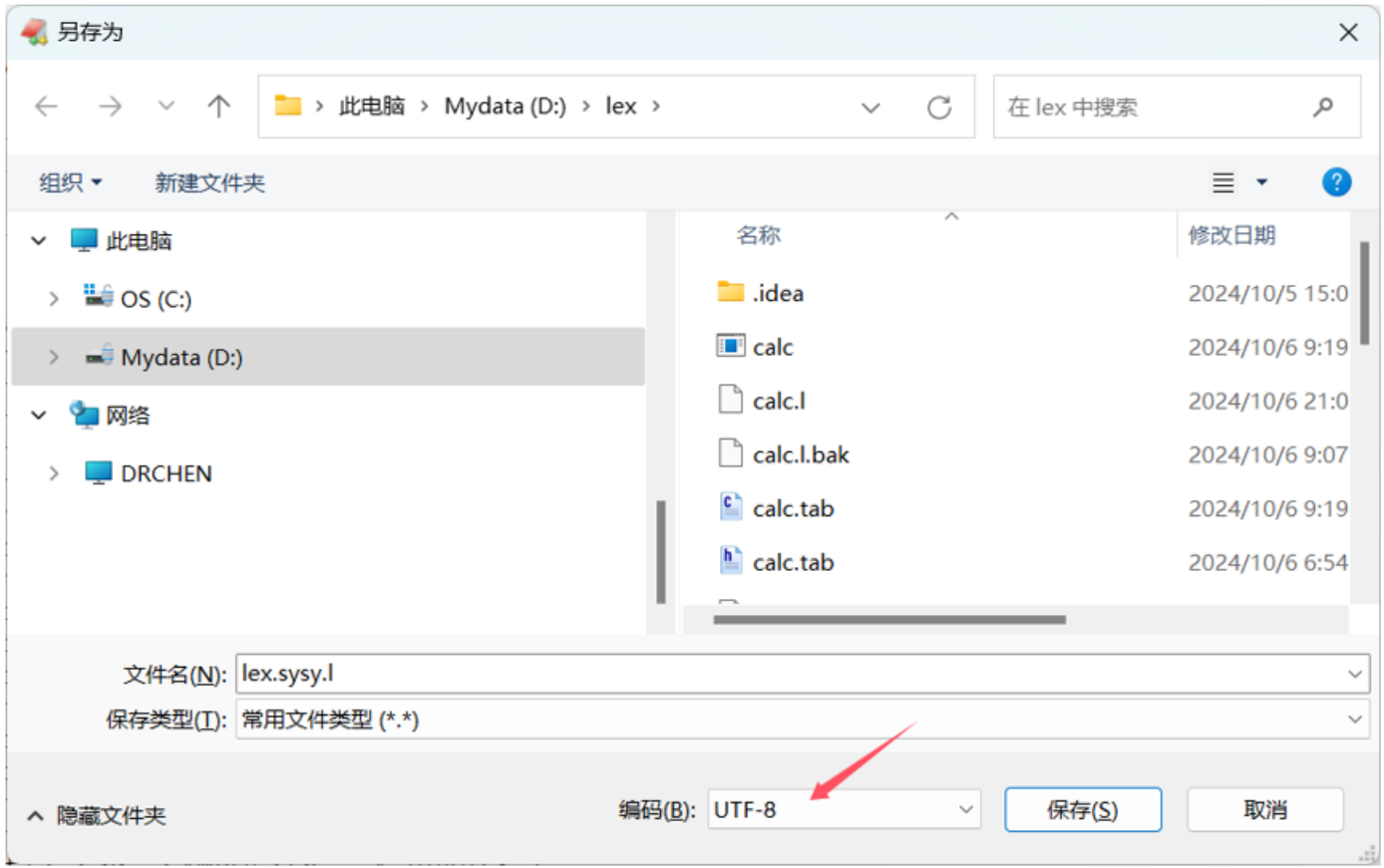
```
78
79  int main(int argc, char *argv[]) {
80      int token;
81      if (argc != 2) {
82          fprintf(stderr, "Usage: %s <input_file>\n", argv[0]);
83          return 1;
84      }
85
86      yyin = fopen(argv[1], "r");
87      if (!yyin) {
88          fprintf(stderr, "Error opening input file\n");
89          return 1;
90      }
91
92      while((token = yylex())!=0){
93          printf("%s    %d",yytext,token);
94          if(token == NUMBER) printf(" =%d\n",yylval);
95          else printf("\n");
96      }
97      fclose(yyin);
98      return 0;
99   }
100
```

1 检查程序运行输入参数，是否有需词法分析的文件名及路径

2 打开需词法分析的文件

3 显示词法分析的结果

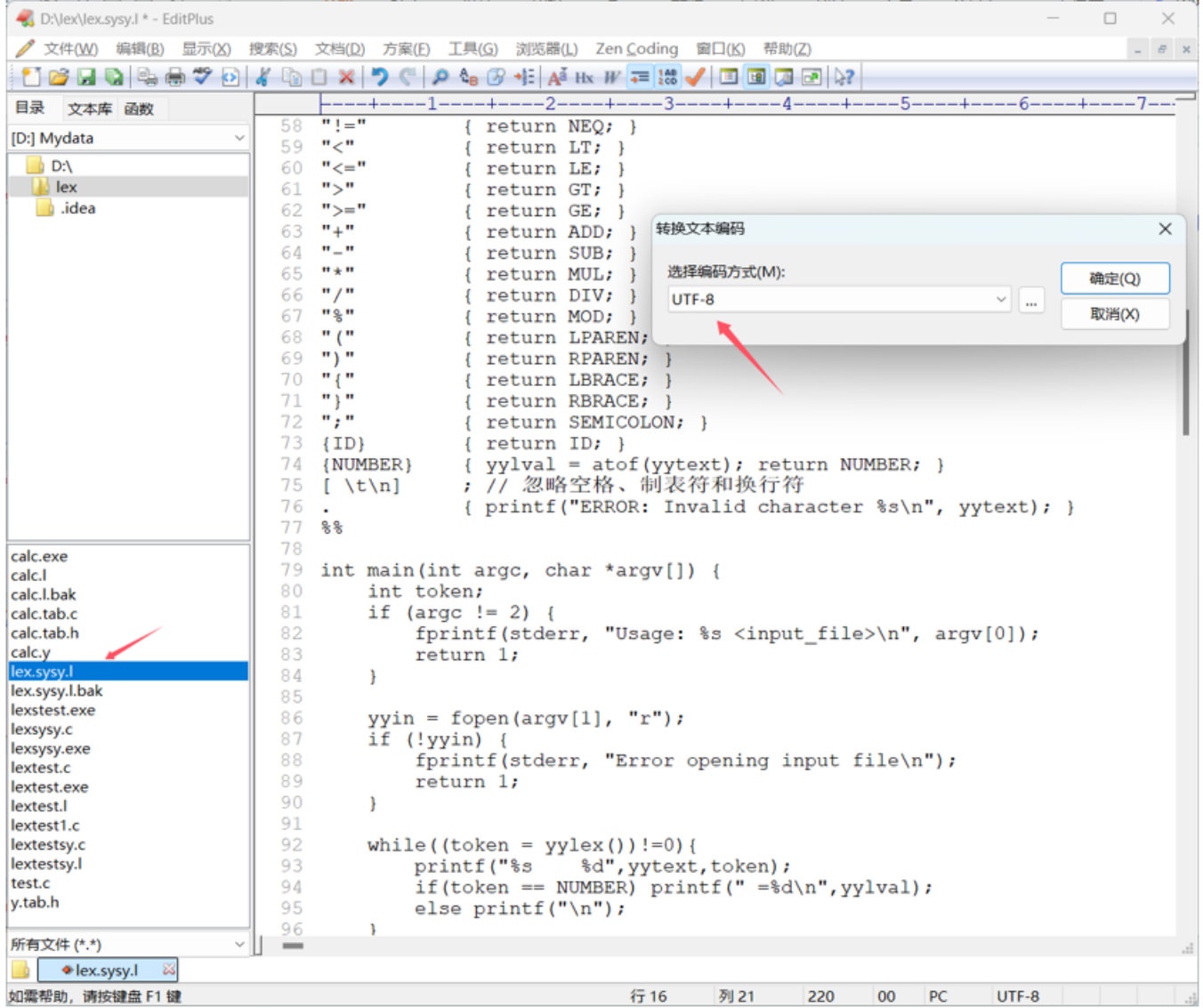
保存编写的程序文件为lex.sysy.l



模式匹配说明

yylex()函数被调用之后，它首先检查全局文件指针变量yyin是否有定义，如有，则将之设置为将要扫描的文件指针。如无，则设置为标准输入文件stdin.同理，如全局文件指针变量yyout无定义，则将之设置为标准输出文件stdout。

保存词法分析程序为lex.sysy.l,字符编码必须选UTF-8



使用flex -o lexsysy.c lex.sysy.l 转化纯C语言程序

```
C:\Users\chena>D:

D:\>
D:\>cd lex

D:\lex>dir
 驱动器 D 中的卷是 Mydata
 卷的序列号是 F27A-34EB

 D:\lex 的目录

2024/10/07  06:49    <DIR>          .
2024/10/05  15:08    <DIR>          .idea
2024/10/06  09:19           109,277 calc.exe
2024/10/06  21:09             1,201 calc.l
2024/10/06  09:07             1,652 calc.l.bak
2024/10/06  09:19            44,340 calc.tab.c
2024/10/06  06:54             2,945 calc.tab.h
2024/10/06  09:17             2,157 calc.y
2024/10/07  06:50             5,566 lex.sysy.l
2024/10/07  06:46             5,589 lex.sysy.l.bak
2024/10/06  12:42           105,844 lexstest.exe
2024/10/07  06:49            53,366 lexsysy.c
2024/10/07  06:49           106,366 lexsysy.exe
2024/10/06  13:41            48,629 lextest.c
2024/10/06  13:42           105,826 lextest.exe
2024/10/06  13:39             1,397 lextest.l
2024/10/06  13:21            48,595 lextest1.c
2024/10/06  13:18            48,663 lextestsy.c
2024/10/06  13:11             1,397 lextestsy.l
2024/10/04  21:56               109 test.c
2024/10/05  18:53             2,519 y.tab.h
              19 个文件        695,438 字节
               2 个目录 318,599,954,432 可用字节

D:\lex>
```

```
命令提示符
 D:\lex 的目录

2024/10/07  06:49    <DIR>          .
2024/10/05  15:08    <DIR>          .idea
2024/10/06  09:19           109,277 calc.exe
2024/10/06  21:09             1,201 calc.l
2024/10/06  09:07             1,652 calc.l.bak
2024/10/06  09:19            44,340 calc.tab.c
2024/10/06  06:54             2,945 calc.tab.h
2024/10/06  09:17             2,157 calc.y
2024/10/07  06:50             5,566 lex.sysy.l
2024/10/07  06:46             5,589 lex.sysy.l.bak
2024/10/06  12:42           105,844 lexstest.exe
2024/10/07  06:49            53,366 lexsysy.c
2024/10/07  06:49           106,366 lexsysy.exe
2024/10/06  13:41            48,629 lextest.c
2024/10/06  13:42           105,826 lextest.exe
2024/10/06  13:39             1,397 lextest.l
2024/10/06  13:21            48,595 lextest1.c
2024/10/06  13:18            48,663 lextestsy.c
2024/10/06  13:11             1,397 lextestsy.l
2024/10/04  21:56               109 test.c
2024/10/05  18:53             2,519 y.tab.h
              19 个文件        695,438 字节
               2 个目录 318,599,954,432 可用字节

D:\lex>flex -lexsysy.c lex.sysy.l
flex: Unrecognized option `e'
Try `flex --help' for more information.

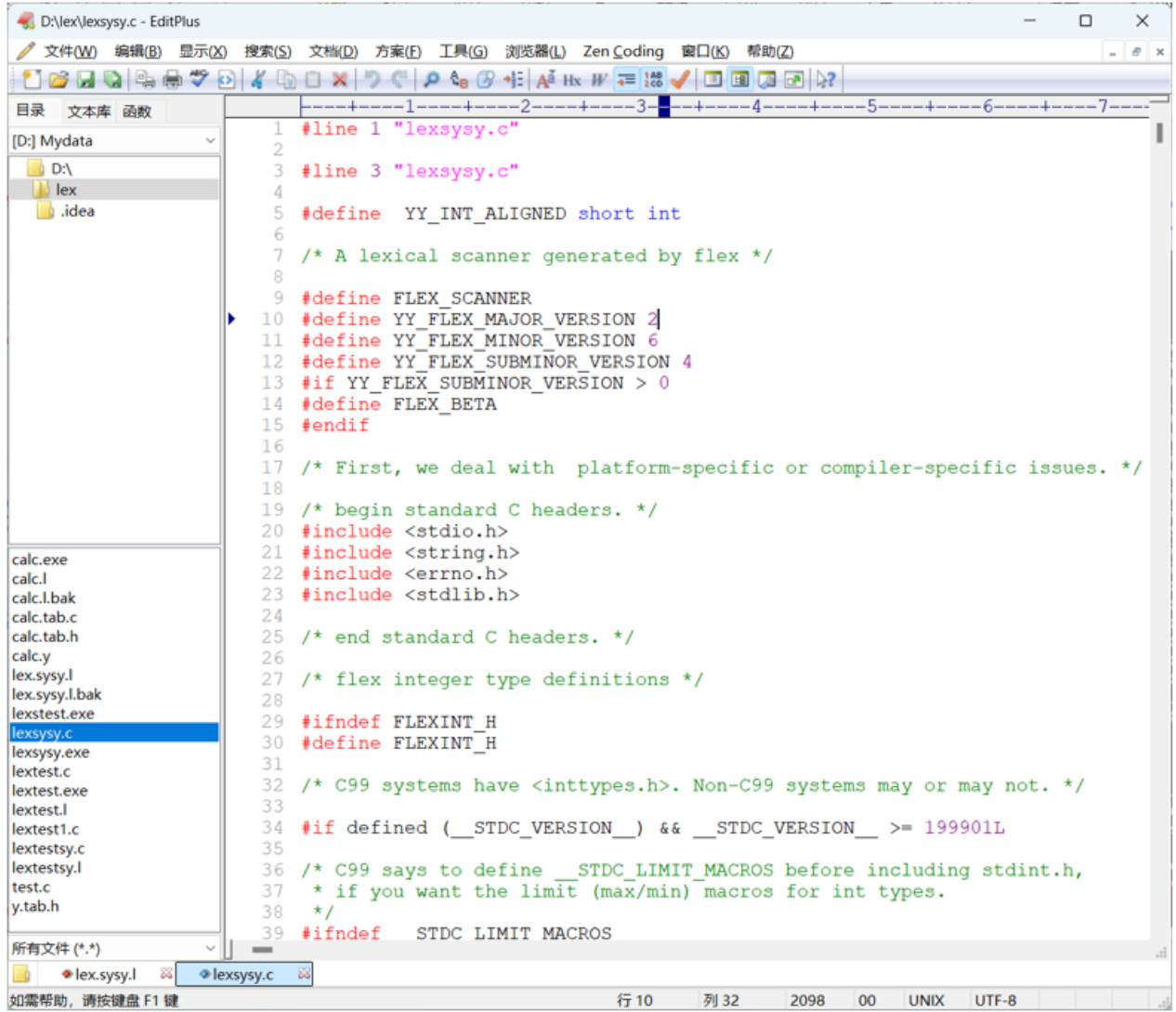
D:\lex>flex -o lexsysy.c lex.sysy.l

D:\lex>
```

1 注意：-o参数后面指出输出文件名

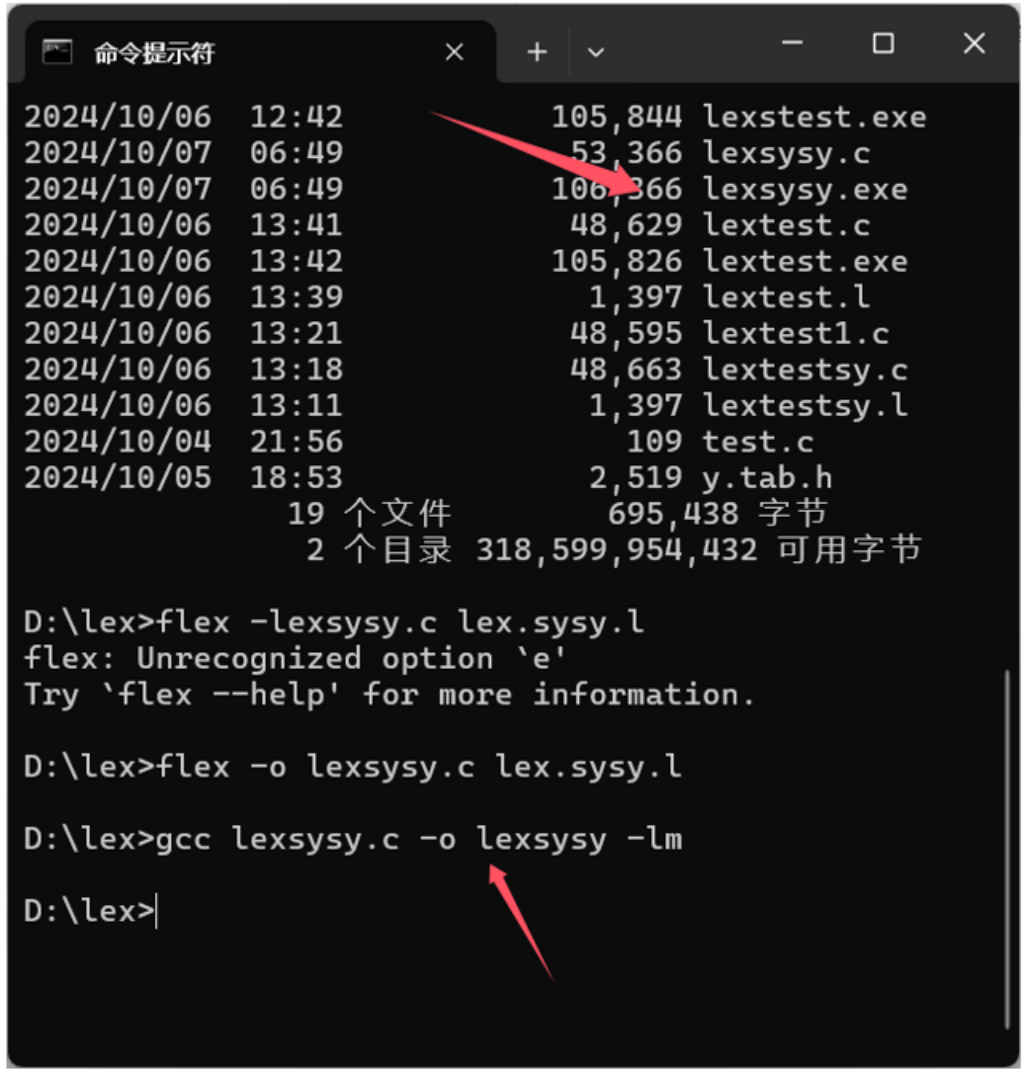
打开lexsysy.c

C语言程序



编译lexsysy.c C语言程序

gcc lexsysy.c -o lexsysy -lm



运行 lexsysy test.c 分析test.c程序

```
1  int main() {
2      int a = 10;
3      if (a > 5) {
4          return a;
5      } else {
6          return 0;
7      }
8  }
```

```
命令提示符

D:\lex>lexsysy test.c
int     256
main     281
(     275
)     276
{     277
int     256
a     281
=     263
10     280 =10
;     279
if     257
(     275
a     281
>     268
5     280 =5
)     276
{     277
return     260
a     281
;     279
}     278
else     258
{     277
return     260
0     280 =0
;     279
}     278
}     278

D:\lex>
```

Flex高级部分：(系统变量)

yyin	FILE* 类型。 它指向 lexer 正在解析的当前文件。
yyout	FILE* 类型。 它指向记录 lexer 输出的位置。 缺省情况下，yyin 和 yyout 都指向标准输入和输出。
yytext	匹配模式的文本存储在这一变量中（char*）。
yyleng	给出匹配模式的长度。
yylineno	提供当前的行数信息。 （lexer不一定支持。）

Flex高级部分：(系统函数)

yylex()	这一函数开始分析。 它由 Lex 自动生成。
yywrap()	这一函数在文件（或输入）的末尾调用。 如果函数的返回值是1，就停止解析。 因此它可以用来解析多个文件。 代码可以写在第三段，这就能够解析多个文件。 方法是使用 yyin 文件指针（见上表）指向不同的文件，直到所有的文件都被解析。 最后，yywrap() 可以返回 1 来表示解析的结束。
yyless(int n)	这一函数可以用来送回除了前n个字符外的所有读出标记。
yymore()	这一函数告诉 Lexer 将下一个标记附加到当前标记后。

样例程序1：

```
1   %{
2   #include <stdio.h>
3   #include "token.h"
4   %}
5
6   int yywrap();
7
8   letter  [A-Za-z_]
9   digit   [0-9]
10  digits  -?{digit}+
11  hexdigit     0(x|X)[0-9a-fA-F]+
12  number       ([+-])?{digit}+(\.{digit}+)?(E[+-]?{digit}+)
13  Identifier   {letter}({letter}|{digit})*
14
```

样例程序1（续）

```
%%
{hexdigit}      { printf("<%d, %d>\n", num_INT, strtol(yytext, NULL, 16)); }
{number}        { printf("<%d, %s>\n", num_FLOAT, yytext); }
{digits}        { printf("<%d, %s>\n", num_INT, yytext); }

[ \t\n]+          ; // Skip whitespace
"int"             { printf("<%d, KEYWORD>\n", Y_INT); }
"float"           { printf("<%d, KEYWORD>\n", Y_FLOAT); }
"void"            { printf("<%d, KEYWORD>\n", Y_VOID); }
"const"           { printf("<%d, KEYWORD>\n", Y_CONST); }
"if"              { printf("<%d, KEYWORD>\n", Y_IF); }
"else"            { printf("<%d, KEYWORD>\n", Y_ELSE); }
"while"           { printf("<%d, KEYWORD>\n", Y_WHILE); }
"break"           { printf("<%d, KEYWORD>\n", Y_BREAK); }
"continue"        { printf("<%d, KEYWORD>\n", Y_CONTINUE); }
"return"          { printf("<%d, KEYWORD>\n", Y_RETURN); }
{Identifier}      { printf("<%d, %s>\n", Y_ID,yytext); }

"+"               { printf("<%d, OPERATOR>\n", Y_ADD); }
"-"               { printf("<%d, OPERATOR>\n", Y_SUB); }
"*"               { printf("<%d, OPERATOR>\n", Y_MUL); }
"/"               { printf("<%d, OPERATOR>\n", Y_DIV); }
"%"               { printf("<%d, OPERATOR>\n", Y_MODULO); }
"<="              { printf("<%d, OPERATOR>\n", Y_LESSEQ); }
"<"               { printf("<%d, OPERATOR>\n", Y_LESS); }
">="              { printf("<%d, OPERATOR>\n", Y_GREATEQ); }
">"               { printf("<%d, OPERATOR>\n", Y_GREAT); }
"!="              { printf("<%d, OPERATOR>\n", Y_NOTEQ); }
"=="              { printf("<%d, OPERATOR>\n", Y_EQ); }
"!"               { printf("<%d, OPERATOR>\n", Y_NOT); }
"&&"              { printf("<%d, OPERATOR>\n", Y_AND); }
"||"              { printf("<%d, OPERATOR>\n", Y_OR); }
"="               { printf("<%d, OPERATOR>\n", Y_ASSIGN); }
```

样例程序1（续）

```
48
49  "("             { printf("<%d, SYMBOL>\n", Y_LPAR); }
50  ")"             { printf("<%d, SYMBOL>\n", Y_RPAR); }
51  "{"             { printf("<%d, SYMBOL>\n", Y_LBRACKET); }
52  "}"             { printf("<%d, SYMBOL>\n", Y_RBRACKET); }
53  "["             { printf("<%d, SYMBOL>\n", Y_LSQUARE); }
54  "]"             { printf("<%d, SYMBOL>\n", Y_RSQUARE); }
55  ","             { printf("<%d, SYMBOL>\n", Y_COMMA); }
56  ";"             { printf("<%d, SYMBOL>\n", Y_SEMICOLON); }
57  "//"            { while (input() != '\n') ;     }
58  "/*"            {int c; while ((c = input()) != '*' || (c = input()) != '/') ; }
59  .               ;
60
61  %%
62
63  void yyerror(const char *s) {
64      fprintf(stderr, "%s\n", s);
65  }
66
67  int yywrap() {
68      return 1;
69  }
70
71
72  int main() {
73      int token;
74      while((token = yylex())!=0){}
75      return 0;
76  }
```

样例程序2：

```
1
2
3  // ----------------------------文--件--分--隔----------------------------
4  // 文件名：lex.sysy.l
5
6  %{
7  #include <stdio.h>
8  #include <stdlib.h>
9  #include <string.h>
10 #include "token.h"
11 %}
12
13 int yywrap();
14
15 letter  [A-Za-z_]
16 digit   [0-9]
17 digits  (-)?{digit}+
18 hexdigit 0[xX][0-9a-fA-F]+
19 number   ([+-])?{digit}+(\.{digit}+)?(E[+-]?{digit}+)
20 Identifier {letter}({letter}|{digit})*
21
22 %{
23     FILE *yyin;
24     int digit_int;
25     float digit_float;
26 %}
```

样例程序2（续）：

```
27
28  %%
29
30  {hexdigit}          {digit_int = strtol(yytext, NULL, 16); return num_INT;}
31  {digits}            {digit_int = atoi(yytext); return num_INT;}
32  {number}            {digit_float = atoi(yytext); return num_FLOAT;}
33
34
35  "int"               {  return Y_INT; }
36  "float"             {  return Y_FLOAT; }
37  "void"              {  return Y_VOID; }
38  "const"             {  return Y_CONST; }
39  "if"                {  return Y_IF; }
40  "else"              {  return Y_ELSE; }
41  "while"             {  return Y_WHILE; }
42  "break"             {  return Y_BREAK; }
43  "continue"          {  return Y_CONTINUE; }
44  "return"            {  return Y_RETURN; }
45  {Identifier}        {  return Y_ID; }
46
```

样例程序2（续）：

```
47  "+"                 {  return Y_ADD; }
48  "-"                 {  return Y_SUB; }
49  "*"                 {  return Y_MUL; }
50  "/"                 {  return Y_DIV; }
51  "%"                 {  return Y_MODULO; }
52  "<="                {  return Y_LESSEQ; }
53  "<"                 {  return Y_LESS; }
54  ">="                {  return Y_GREATEQ; }
55  ">"                 {  return Y_GREAT; }
56  "!="                {  return Y_NOTEQ; }
57  "=="                {  return Y_EQ; }
58  "!"                 {  return Y_NOT; }
59  "&&"                {  return Y_AND; }
60  "||"                {  return Y_OR; }
61  "="                 {  return Y_ASSIGN; }
62
```

样例程序2（续）：

```
63  "("                 {  return Y_LPAR; }
64  ")"                 {  return Y_RPAR; }
65  "{"                 {  return Y_LBRACKET; }
66  "}"                 {  return Y_RBRACKET; }
67  "["                 {  return Y_LSQUARE; }
68  "]"                 {  return Y_RSQUARE; }
69  ","                 {  return Y_COMMA; }
70  ";"                 {  return Y_SEMICOLON; }
71
72  [ \t\n]+            ; // Skip whitespace
73  "//"                { while (input() != '\n') ;     }
74  "/*"                { int c; while ((c = input()) != '*' || (c = input()) != '/') ; }
75  .                   ;
76
77  %%
78
79  void yyerror(const char *s){
80      fprintf(stderr, "%s\n", s);
81  }
82
83  int yywrap() {
84      return 1;
85  }
86
```

```c
int main(int argc, char *argv[]) {
    int token;
    if (argc == 2) {
        yyin = fopen(argv[1], "r");
        if (!yyin) {
           fprintf(stderr, "Error opening input file\n");
           return 1;
        }
    }
    else if (argc >=3 ) {
        return 1;
    }
    while(token = yylex()){
        if ((token>=Y_INT && token<=Y_RETURN)||token==Y_FLOAT)
          printf("<%d, KEYWORD>\n",token);
        else if (token>=Y_ADD && token<=Y_ASSIGN)
          printf("<%d, OPERATOR>\n",token);
        else if (token>=Y_LPAR && token<=Y_SEMICOLON)
          printf("<%d, SYMBOL>\n", token);
        else if (token == Y_ID)
          printf("<%d, %s>\n", Y_ID,yytext);
        else if (token == num_INT)
          printf("<%d, %d>\n", token, digit_int);
        else if (token == num_FLOAT)
          printf("<%d, %f>\n", token, digit_float);
    }
    fclose(yyin);
    return 0;
 }
```